

Laporan Tugas Besar 3

IF2211 - Strategi Algoritma

Implementasi Pattern Matching pada Chromium Browser
Extension untuk Deteksi Judi Online



Disusun oleh:

Kelompok : GDP-UOB-OCBC

Muhamad Hasbullah Faris 18223014

Stanislaus Ardy Bramantyo 18223057

Nazwan Siddqi Muttaqin 18223066

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

Daftar Isi

Daftar Isi.....	2
I. Deskripsi Tugas.....	4
II. Landasan Teori.....	5
2.1 Penjelasan Pattern Matching.....	5
2.2 Algoritma Knuth-Morris-Pratt (KMP).....	5
2.3 Algoritma Boyen-Moore (BM).....	5
2.4 Algoritma Aho-Corasick.....	5
2.5 Algoritma Rabin Karp.....	5
2.6 Penjelasan Browser Extension.....	5
III. Analisis Pemecahan Masalah.....	6
3.1 Langkah-Langkah Pemecahan Masalah.....	6
3.2 Proses Pemetaan Masalah.....	6
3.3 Fitur Fungsional dan Arsitektur Extension.....	6
3.4 Contoh Ilustrasi Kasus.....	6
IV. Implementasi dan pengujian.....	7
4.1 Implementasi Program.....	7
4.1.1 Algoritma String Matching.....	7
4.1.2 Algoritma Regular Expression (RegEx).....	7
4.1.3 Chromium Browser Extension.....	7
4.2 Penjelasan Tata Cara Penggunaan Extension.....	7
4.3 Hasil Pengujian.....	7
4.4 Analisis Efisiensi dan Perbandingan.....	7
V. Kesimpulan dan Saran.....	8
5.1 Kesimpulan.....	8
5.2 Saran.....	9
VI. Lampiran.....	11
VII. Pernyataan Kejujuran.....	13
VIII. Daftar Pustaka.....	14

I. Deskripsi Tugas

Pada Tugas Besar 3 IF2211 Strategi Algoritma ini, kami ditugaskan untuk mengimplementasikan algoritma pattern matching dalam bentuk Chromium browser extension untuk mendeteksi konten yang mengandung unsur judi online. Konten judi online sering ditemukan pada halaman web dengan variasi penulisan tertentu, seperti penggabungan kata dan angka, misalnya GACOR99, MAXWIN88, SLOT777, dan MADU308. Selain itu, beberapa kata juga dapat dimodifikasi menggunakan karakter yang mirip secara visual, seperti H0KI88 atau G4COR99.

Extension yang dibangun diberi nama Judol Detector. Sistem ini bekerja dengan membaca teks pada halaman web aktif, kemudian mencocokkannya dengan daftar kata kunci yang tersimpan pada file keywords.txt. Proses pencocokan dilakukan menggunakan algoritma Knuth-Morris-Pratt (KMP) dan Boyer-Moore (BM) yang diimplementasikan secara manual, serta Regular Expression (RegEx) untuk mendeteksi pola <kata><angka>. Selain itu, sistem juga menerapkan Weighted Levenshtein Distance untuk mendukung fuzzy matching terhadap kata yang tidak sama persis, tetapi masih memiliki kemiripan dengan kata kunci.

Secara umum, sistem yang dibangun memiliki alur sebagai berikut:

1. **Input:** Sistem membaca teks dari elemen DOM pada halaman web aktif dan daftar kata kunci dari file keywords.txt.
2. **Pemrosesan:** Sistem melakukan pencocokan string menggunakan KMP, Boyer-Moore, RegEx, dan Weighted Levenshtein Distance. Hasil pencocokan kemudian dikumpulkan beserta informasi algoritma, jumlah kemunculan, dan waktu eksekusi.
3. **Output:** Teks yang terdeteksi sebagai konten judi online akan diberi highlight pada halaman web. Saat pengguna melakukan hover pada teks tersebut, sistem menampilkan tooltip berisi informasi hasil deteksi. Selain itu, popup extension menampilkan statistik realtime seperti total keyword yang ditemukan, jumlah match tiap algoritma, dan waktu eksekusi.

Dengan demikian, tugas besar ini bertujuan untuk menerapkan konsep pattern matching ke dalam aplikasi nyata yang dapat mendeteksi pola teks tertentu secara otomatis pada halaman web melalui browser extension.

II. Landasan Teori

2.1 Penjelasan Pattern Matching

Pattern matching atau pencocokan string adalah proses untuk mencari keberadaan suatu pola tertentu di dalam sebuah teks. Dalam konsep pencocokan string, terdapat dua komponen utama, yaitu teks dan pattern. Teks atau T merupakan string utama dengan panjang n karakter, sedangkan pattern atau P merupakan string yang lebih pendek dengan panjang m karakter dan akan dicari kemunculannya di dalam teks. Tujuan utama dari pattern matching adalah menemukan posisi di dalam teks yang bersesuaian dengan pattern tersebut.

Secara umum, pattern matching banyak digunakan pada berbagai aplikasi komputasi, seperti pencarian kata pada editor teks, mesin pencari web, analisis citra, dan bioinformatika. Pada editor teks, algoritma pencocokan string digunakan untuk menemukan kata tertentu di dalam dokumen. Pada mesin pencari web, konsep ini digunakan untuk menemukan halaman atau dokumen yang relevan dengan kata kunci yang diberikan pengguna. Selain itu, pencocokan pola juga dapat digunakan dalam analisis citra maupun pencocokan rantai asam amino pada bidang bioinformatika.

Pendekatan paling sederhana dalam pattern matching adalah algoritma brute force. Algoritma ini bekerja dengan memeriksa setiap kemungkinan posisi di dalam teks untuk mengetahui apakah pattern dimulai pada posisi tersebut. Pattern akan digeser satu karakter demi satu karakter hingga ditemukan kecocokan atau hingga seluruh teks selesai diperiksa. Meskipun sederhana, pendekatan ini dapat menjadi kurang efisien ketika teks dan pattern berukuran besar karena banyak perbandingan karakter yang dilakukan secara berulang.

Dalam analisis kompleksitasnya, brute force memiliki kompleksitas waktu terburuk sebesar $O(mn)$, dengan m sebagai panjang pattern dan n sebagai panjang teks. Kasus terbaiknya dapat mencapai $O(n)$, yaitu ketika karakter pertama pattern tidak pernah sama dengan karakter pada teks yang sedang dibandingkan. Sementara itu, pada kasus rata-rata untuk teks biasa, proses pencarian sering kali dapat berjalan cukup cepat.

Pada Tugas Besar 3 ini, konsep pattern matching digunakan sebagai dasar dari sistem pendeteksi konten judi online pada halaman web. Teks yang terdapat pada elemen DOM halaman web diperlakukan sebagai teks utama, sedangkan kata kunci judi online seperti GACOR, MAXWIN, SLOT, dan TOGEL diperlakukan sebagai pattern yang ingin dicari. Untuk meningkatkan efisiensi dan akurasi, sistem tidak hanya menggunakan sederhana, tetapi juga menerapkan beberapa algoritma pattern matching, seperti Knuth-Morris-Pratt, Boyer-Moore, RegEx, Aho-Corasick, dan Rabin-Karp. Dengan pendekatan ini, sistem dapat mengenali pola kata yang muncul secara langsung maupun variasi penulisan tertentu yang umum digunakan pada konten judi online.

2.2 Algoritma Knuth-Morris-Pratt (KMP)

Algoritma Knuth-Morris-Pratt (KM) adalah sebuah algoritma *string matching* yang digunakan untuk mencari sebuah *pattern* di sebuah *text* secara efisien. Berbeda dengan pendekatan *naive* yang mengulang pencarian dari awal setiap kali terjadi ketidakcocokan, KMP akan memanfaatkan informasi dari kecocokan sebelumnya untuk menghindari perbandingan yang tidak perlu. Hal ini memungkinkan pencarian dilakukan tanpa perlu mundur pada posisi *text*.

Struktur utama yang digunakan dalam algoritma ini adalah **failure function** (disebut juga tabel LPS), yang simpelnya adalah sebuah *array* yang menyimpan panjang *prefix* terpanjang dari *pattern* yang sekaligus merupakan *suffix*-nya. Dengan tabel tersebut, ketika terjadi ketidakcocokan pada posisi tertentu, algoritma tidak perlu kembali ke awal *pattern*, melainkan sudah cukup lompat ke posisi yang ditunjuk nantinya oleh *failure function*.

Secara algoritmik, Knuth-Morris-Pratt dapat dijelaskan menjadi:

- I. **Inisialisasi:** Bangun *failure function* ($f[]$) dari sebuah *pattern* P .
- II. **Pencarian:** Telusuri *text* T dari kiri ke kanan dengan pointer i , dan itu akan dicocokkan terhadap *pattern* dengan pointer j .
- III. **Kecocokan Karakter:** IF $T[i] == P[j]$, maka $i++$ dan $j++$.
- IV. **Pattern Ditemukan:** Saat sebuah *pattern* ditemukan, pencarian tetap dilakukan dengan memulai kembali di $j = f[j - 1]$.
- V. **Missmatch:**
 - A. $j > 0$, maka ganti ke $j = f[j - 1]$.
 - B. $j = 0$, maka cukup dengan $i++$ (*starting point* nambah 1).

Algoritma KMP cocok digunakan ketika hanya satu *pattern* yang perlu dicocokkan terhadap *text* secara berulang (e.g. pengecekan kata GACOR saja atau MAXWIN saja). Keunggulan utamanya muncul karena adanya *pointer text* yang tidak pernah mundur, sehingga pencarian tetap linear. Namun, untuk mendekteksi banyak kata kunci sekaligus, KMP perlu dijalankan berulang kali, dalam arti satu kali per *pattern*, sehingga nanti total *complexity* dalam pencarian menjadi k lebih banyak. Sebagai catatan, kompleksitas dari KMP dapat dibagi menjadi 2 (dua), yaitu:

a. Pembangunan Failure Function	$O(m)$
b. Pencarian (1) Pattern di Text	$O(n \times 1)$
Total Complexity	$O(n + m)$

2.3 Algoritma Boyen-Moore (BM)

Algoritma Boyer-Moore adalah sebuah algoritma *string matching* satu *pattern* yang bekerja dengan prinsip yang berlawanan dari KMP, yaitu pencocokan dilakukan dari kanan ke arah kiri pada *pattern*. Kunggulan dari BM adalah kemampuan untuk melompati banyak karakter sekaligus saat terjadi *mismatch* nantinya, sehingga algoritma ini umumnya lebih cepat dari $O(n)$.

Algoritma BM akan menggunakan 2 (dua) aturan untuk menentukan seberapa jauh *pattern* digeser saat terjadi sebuah *mismatch*:

1. Bad Character Rule

Saat terjadi sebuah *mismatch* di posisi i di *text*, lihat karakter *text* yang menyebabkan *mismatch* tersebut (disebut juga **bad character**). Geser *pattern* sehingga kemunculan terakhir karakter itu di dalam *pattern* sejajar dengan posisi *mismatch*. Pergeseran menggunakan rumus:

$$Shift = j - j[char_text]$$

2. Good Suffix Rule

Saat terjadi *mismatch* setelah beberapa karakter cocok (alias *suffix* dari *pattern* sudah *match*), geser *pattern* sehingga *suffix* yang sudah cocok itu sejajar dengan kemunculannya yang lain di dalam *pattern*. Jika tidak ada, cari *prefix pattern* yang cocok dengan *suffix* tersebut.

$$Shift = j[start_sf] - j[start_sf_other]$$

Secara algoritmik, Knuth-Morris-Pratt dapat dijelaskan menjadi:

1. **Pencocokan**: Sejajarkan *pattern* dengan awal *text*.
2. **Scan R-to-L**: Bandingkan karakter *pattern* p dari posisi paling kanan.
3. **Mismatch**: Hitung *shift* dari 2 (dua) aturan*, baik **Bad Character** & **Good Suffix**, dan gunakan yang paling besar. *Aturan tsb hanya untuk perhitungan *shift*.
4. **Full Match**: Catat posisi, dan lakukan *shift* sesuai aturan *Good Suffix*.
5. **Iterate**: sampai *pattern* sudah sampai/melewati ujung *text*.

Algoritma Boyer-Moore paling cocok digunakan jika *searching* dilakukan **satu pattern dalam satu waktu** terhadap *text* yang panjang. Algoritma ini juga didukung dengan seberapa efisien melakukan *searching* dengan $O(n/m)$ untuk *best casenya*.

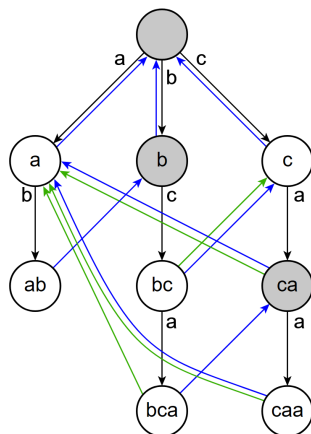
2.4 Algoritma Aho-Corasick

Algoritma Aho-Corasick adalah algoritma string matching yang digunakan untuk mencari banyak pattern sekaligus di dalam sebuah teks. Berbeda dengan KMP dan Boyer-Moore yang umumnya mencocokkan satu pattern terhadap teks, Aho-Corasick membangun struktur automaton dari seluruh daftar pattern terlebih dahulu. Struktur ini memungkinkan proses pencarian dilakukan hanya dengan satu kali pemindaian terhadap teks.

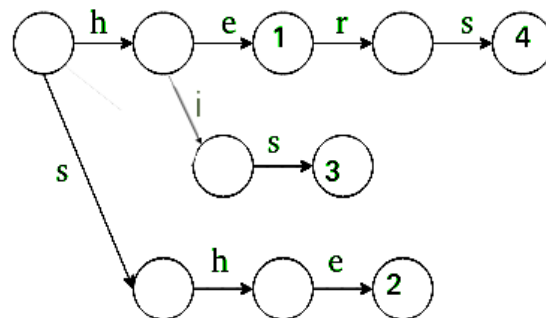
Struktur utama yang digunakan dalam algoritma ini adalah trie, yaitu pohon karakter yang menyimpan seluruh pattern. Selain itu, algoritma ini juga menggunakan failure link, yaitu jalur alternatif yang digunakan ketika terjadi ketidakcocokan karakter. Dengan adanya failure link, proses pencarian tidak perlu diulang dari awal, tetapi dapat berpindah ke state lain yang masih memungkinkan terjadinya kecocokan.

Secara algoritmik, tahapan Aho-Corasick dapat diurai menjadi:

1. **Inisialisasi:** Masukkan seluruh pattern ke dalam struktur trie.
2. **Pembentukan failure link:** Bangun jalur fallback untuk setiap node agar algoritma dapat berpindah ketika terjadi mismatch.
3. **Pencarian:** Telusuri teks dari kiri ke kanan dengan mengikuti transisi pada automaton.
4. **Output:** Jika state yang dikunjungi memiliki pattern yang selesai, maka pattern tersebut dinyatakan ditemukan.



Trie for Arr[] = { he , she, his , hers }



Keunggulan utama Aho-Corasick adalah kemampuannya mencocokkan banyak pattern dalam satu kali proses traversal teks. Hal ini sesuai untuk sistem Judol Detector karena daftar kata kunci judi online dapat berisi banyak kata, seperti GACOR, SLOT, MAXWIN, dan sebagainya. Jika panjang teks adalah n dan total panjang seluruh pattern adalah m , maka tahap pembangunan struktur membutuhkan $O(m)$, sedangkan pencarian dapat dilakukan secara linear terhadap panjang teks ditambah jumlah hasil yang ditemukan.

2.5 Algoritma Rabin Karp

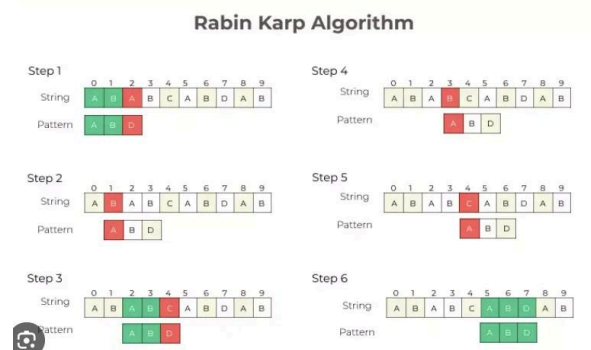
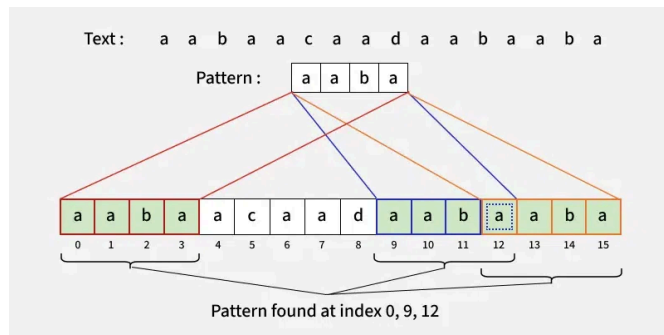
Algoritma Rabin-Karp adalah algoritma string matching yang menggunakan konsep hashing. Pada algoritma ini, pattern dan potongan teks dengan panjang yang sama diubah terlebih dahulu menjadi nilai hash. Jika nilai hash berbeda, maka potongan teks tersebut pasti bukan pattern yang dicari. Namun, jika nilai hash sama, algoritma tetap melakukan verifikasi karakter untuk memastikan kecocokan sebenarnya.

Konsep penting pada Rabin-Karp adalah rolling hash, yaitu teknik menghitung hash untuk window berikutnya secara efisien. Ketika window bergeser satu karakter ke kanan, hash baru tidak dihitung dari awal, tetapi diperbarui dari hash sebelumnya dengan menghapus karakter lama dan menambahkan karakter baru. Rolling hash memang umum digunakan dalam Rabin-Karp untuk mempercepat perhitungan hash substring yang bergerak sepanjang teks.

Secara algoritmik, tahapan Rabin-Karp dapat diurai menjadi:

1. **Inisialisasi:** Hitung nilai hash dari pattern.
2. **Pembentukan window:** Ambil substring awal dari teks dengan panjang yang sama seperti pattern.
3. **Pencocokan hash:** Bandingkan hash window dengan hash pattern.
4. **Verifikasi:** Jika hash sama, lakukan pengecekan karakter satu per satu.
5. **Pergeseran:** Geser window ke kanan dan hitung hash baru menggunakan rolling hash.

6. **Basis:** Ulangi proses hingga seluruh teks selesai diperiksa.



Keunggulan Rabin-Karp adalah efisiensinya dalam melakukan pencocokan awal melalui nilai hash. Pada kondisi rata-rata, algoritma ini dapat berjalan cukup cepat karena sebagian besar substring dapat dieliminasi hanya dari perbedaan nilai hash. Namun, jika terjadi banyak hash collision, algoritma tetap perlu melakukan verifikasi karakter berulang sehingga kompleksitas terburuknya dapat mendekati $O(nm)$. Dalam Judol Detector, Rabin-Karp digunakan sebagai algoritma alternatif untuk mendeteksi kata kunci judi online secara exact matching dengan pendekatan berbasis hash.

2.6 Penjelasan Browser Extension

Browser extension adalah sebuah perangkat lunak yang berjalan di dalam peramban untuk menambah fungsionalitas tertentu. Extension mampu mengakses halaman web, memodifikasi tampilan, serta berinteraksi dengan pengguna melalui antarmuka tambahan. Dalam konteks ini, extension digunakan sebagai lapisan yang memproses konten halaman secara otomatis dan menampilkan hasil deteksi secara real time. Extension terdiri dari beberapa komponen utama yang saling berkomunikasi melalui mekanisme messaging, dan perilakunya diatur oleh sebuah berkas konfigurasi.

Alur kerja extension dimulai saat halaman dibuka. Browser memuat content script sesuai konfigurasi manifest. Content script mengekstrak teks dari DOM dan melakukan pemrosesan sesuai kebutuhan, lalu menampilkan hasil di halaman. Jika

pengguna mengubah pengaturan di popup, perubahan dikirim ke background dan disimpan ke storage, kemudian diteruskan ke content script agar perilaku deteksi menyesuaikan konfigurasi terbaru. Dengan alur ini, extension dapat bekerja secara konsisten, aman, dan efisien di berbagai halaman web.

III. Analisis Pemecahan Masalah

3.1 Langkah-Langkah Pemecahan Masalah

Secara umum, sistem melakukan pencarian pola pada konten halaman web yang telah diproses menjadi bentuk teks agar mudah dianalisis. Proses ini dilakukan untuk menemukan kemunculan kata kunci secara tepat, serta menangani variasi penulisan yang tidak selalu sama dengan bentuk literalnya. Adapun tahapan pencocokan yang diimplementasikan adalah sebagai berikut.

1. Inisialisasi fungsi-fungsi ataupun pra-pemrosesan yang dibutuhkan untuk algoritma yang dipilih, seperti penyusunan tabel LPS pada KMP atau tabel bad-character pada Boyer–Moore.
2. Lakukan perbandingan karakter antara pola dan teks menggunakan algoritma yang dipilih.
3. Jika terjadi ketidakcocokan, penanganan karakter pattern akan menyesuaikan algoritma yang dipilih. Untuk KMP dan BM akan dilakukan pergeseran sesuai aturan masing-masing.
4. Jika terjadi kecocokan, maka keyword akan ditampung sebagai hasil temuan beserta posisi kemunculannya untuk dipakai pada proses highlight/tooltip.
5. Ulangi proses perbandingan (langkah 2), penanganan ketidakcocokan (langkah 3), dan pencatatan temuan (langkah 4) hingga seluruh teks selesai ditelusuri.
6. Jika terdapat pola yang tidak dapat ditangkap dengan keyword literal, maka akan dijalankan pencocokan berbasis regex untuk mendeteksi variasi penulisan.
7. Kembalikan total temuan beserta posisi kemunculan sebagai hasil akhir.

3.2 Proses Pemetaan Masalah

Sebelum masalah dipetakan menjadi elemen permasalahan, didefinisikan terlebih dahulu objek dasar untuk memudahkan proses abstraksi.

1. Teks Halaman

Teks halaman adalah representasi data utama yang akan dianalisis oleh sistem. Konten diambil dari DOM halaman web, kemudian digabungkan menjadi string tunggal. Representasi linear ini diperlukan agar algoritma string matching dapat bekerja secara efisien pada data yang semula bersifat visual dan terstruktur sebagai elemen HTML.

2. Kata Kunci

Kata kunci merepresentasikan kriteria atau pola yang ingin dideteksi. Dalam algoritma, kata kunci berperan sebagai **pattern** yang dicocokkan terhadap teks halaman. Pengguna dapat memasukkan satu atau lebih kata kunci sekaligus. Interaksi antara kata kunci dan teks diatur oleh dua aturan pencocokan:

- **Exact Match**

Pada pencocokan ini, setiap kata kunci akan dicocokkan secara sempurna. Pencocokan dilakukan pada **string**, bukan word; sehingga substring tetap dianggap *match* meskipun berada di tengah kata.

- **Fuzzy Match**

Berbeda dari *exact match*, Jika keyword *literal* tidak ditemukan, pola regex dipakai untuk menangkap variasi penulisan atau format tertentu (misalnya angka).

Setelah objek dasar didefinisikan, dilakukan pemetaan ke elemen formal algoritma string matching, dimana dipetakan menjadi:

1. **Text:** Pada sistem ini, teks yang dimaksud adalah konten halaman web yang sudah dinormalisasi.
2. **Pattern:** Objek kata kunci yang dimasukkan pengguna yang akan berperan sebagai “pola” yang akan dilakukan pencocokan.

Alur Pencocokan dijabarkan sebagai berikut:

1. **Inisialisasi:** Sistem akan menginisialisasi algoritma pencocokan yang dipilih pengguna, baik KMP maupun BM.
2. **Pemindaian:** Setiap kata kunci akan dicocokkan dengan teks halaman secara menyeluruh, dari awal hingga akhir, untuk mendeteksi semua kemunculan pola di dalam teks.
3. **Hasil:** Keluaran dari proses ini adalah frekuensi kemunculan masing-masing kata kunci pada halaman. Informasi ini digunakan sebagai indikator utama konten yang perlu ditandai (*highlight*) atau disamarkan (*blur*).

3.3 Fitur Fungsional dan Arsitektur Extension

3.3.1 Fitur Fungsional

Adapun fitur fungsional pada aplikasi ini adalah sebagai berikut.

1. **Deteksi konten** berbasis keyword (KMP/BM) dan pola (Regex).
2. **Highlight & tooltip** untuk menandai hasil deteksi.
3. **Blur konten** sebagai perlindungan visual.
4. **Popup UI** untuk pengaturan dan ringkasan informasi.

3.3.2 Arsitektur Aplikasi

Secara keseluruhan, sistem dibagi menjadi empat komponen utama, yaitu halaman web, content script, background service worker, dan popup UI. Ketika pengguna mengakses halaman, content script berjalan untuk mengekstraksi teks dan menjalankan proses pencocokan. Background service worker mengatur *lifecycle extension* dan komunikasi antar-komponen, sedangkan popup UI menjadi antarmuka pengguna untuk pengaturan. Penyimpanan lokal digunakan untuk menyimpan preferensi dan daftar keyword agar tetap konsisten di setiap sesi.

1. **Frontend**

Pada sisi frontend, extension menyediakan popup UI sebagai antarmuka utama pengguna. Dengan popup UI, pengguna dapat

melakukan konfigurasi tanpa mengganggu tampilan halaman web yang sedang dibaca.

2. Interaksi Antar-Komponen dan Storage

Interaksi utama terjadi antara popup UI, background service worker, dan content script melalui mekanisme *messaging*. Ketika pengguna mengubah pengaturan di popup, perubahan tersebut disimpan ke storage, lalu content script membaca konfigurasi terbaru untuk menentukan proses highlight/tooltip dan blur. Alur ini menjaga konsistensi perilaku extension dan memastikan hasil deteksi sesuai preferensi pengguna.

3.4 Contoh Ilustrasi Kasus

Sebagai contoh kasus, di sini diandaikan bahwa pengguna mengakses artikel promosi yang berisi klaim “bonus instan”, “diskon besar”, dan tautan pendaftaran. Pada halaman tersebut, beberapa kata kunci muncul di judul, paragraf utama, serta tombol ajakan tindakan. Pengguna mengaktifkan fitur blur dan memilih algoritma pencocokan KMP melalui popup. Pada kasus ini, sistem akan memproses halaman web dengan alur sebagai berikut.

1. **Ekstraksi teks:** Content script mengekstrak teks dari elemen DOM, termasuk judul, paragraf, dan teks pada tombol.
2. **Normalisasi:** Teks diubah menjadi lowercase dan dibersihkan dari karakter yang tidak relevan agar konsisten untuk pencocokan.
3. **Pencocokan:** KMP/BM mendeteksi kemunculan keyword literal seperti “bonus” dan “diskon”. Sedangkan RegEx akan menangkap variasi penulisan seperti “bOnus”, “diskOn 50%”, atau format angka yang disisipkan simbol.
4. **Output:** Elemen yang memuat kata kunci di-highlight dan tooltip muncul berisi ringkasan. Apabila fitur blur aktif, blok konten terkait disamarkan.

IV. Implementasi dan pengujian

4.1 Implementasi Program

4.1.1 Algoritma *String Matching*

1. [kmp.ts](#)

```
export interface KMPResult {
  indices: number[];
  comparisons: number;
}

function buildFailure(pattern: string): number[] {
  const n = pattern.length;
  const fail = new Array <number>(n).fill(0); // declare with
  0s

  let k = 0
  for (let i = 1; i < n; i++) {
    while ( (k > 0) && (pattern[k] !== pattern[i]) ) {
      k = fail[k-1];
    }

    if (pattern[k] == pattern[i]) {
      k++;
    }

    fail[i] = k
  }
  return fail
}

// call once at load, pass result into kmpSearch later
export function precompute(pattern: string): number[] {
```

```

    return buildFailure(pattern);
}

export function kmpSearch(text: string, pattern: string,
failure?: number[]): KMPResult {
    const n = text.length;
    const m = pattern.length;

    if ( (m === 0) || m > n) {
        return { indices: [], comparisons: 0 };
    }

    let fail;

    if (failure === null) {
        fail = buildFailure(pattern);
    } else if (failure === undefined) {
        fail = buildFailure(pattern);
    } else {
        fail = failure;
    }

    const indices: number[] = [];

    let comparisons = 0;
    let j = 0;

    for (let i=0; i<n; i++) {
        while ( (j>0) && (pattern[j] !== text[i])) {
            comparisons++;
            j = fail[j-1];
        }
        comparisons++;

        if (pattern[j] === text[i]) {

```



```

        j++;
    }
    if (j === m) {
        indices.push(i - m + 1);
        j = fail[j - 1];
    }
}

return {indices, comparisons};
}

```

Konsep dasar dari algoritma KMP adalah memanfaatkan informasi dari pola itu sendiri. Ketika nanti terjadi ketidakcocokan atau **mismatch** di antara teks dan pola, algoritma akan menggunakan **failure function** untuk menentukan seberapa jauh pencocokan dapat dilanjutkan.

a. Pembangunan *Failure Function*

Fungsi pembangunan *failure function* akan memanfaatkan pola itu sendiri. Sederhananya, nilai dari tabel akan diartikan sebagai panjang *prefix* sekaligus *suffix* terpanjang, tentu ini tidak termasuk *string* itu sendiri.

Akan ada dua indeks yang bergerak, yaitu i yang terus ‘eksplorasi’ dan k sebagai penanda *prefix*. $i = 1$ dan terus bertambah 1 (satu) hingga akhir dari pola. $k = 0$ dan bertambah 1 (satu) jika $\text{pattern}[i] == \text{pattern}[k]$. Namun jika berbeda ($\neq$), maka $k = 0$ kembali. *Value fail function* tiap huruf di pola akan sama dengan nilai k terbaru. Ini dengan catatan *value fail[0]* akan pasti **0**.

b. Pencarian Pola dengan KMP

Dengan tabel *fail function* sudah didefinisikan, dapat kita mengikuti algoritma yang ada, yaitu:

Bandingkan `text[i]` dengan `pattern[j]`.

- I. IF **sama**, `j++` sampai di akhir (`j = fail[i]`).
- II. **ELSE**,
 - i. IF `j > 0`, ubah nilai `j = fail[j-1]`.
 - ii. **ELSE**, cukup `i++`.

Nilai `i` akan selalu bertambah 1 (satu) setiap 'iterasi'nya di dalam algoritma KMP..

2. [boyer-moore.ts](#)

```
export interface BMResult {
  indices: number[];
  comparisons: number;
}

function buildLastOccurrence(pattern: string): Map<string, number> {
  const last = new Map<string, number>();
  for (let i=0; i<pattern.length; i++) {
    last.set(pattern[i], i);
  }

  return last
}

export function precompute(pattern: string): Map<string, number>
{
  return buildLastOccurrence(pattern);
}
```

```

export function bmSearch(text: string, pattern: string, lastOcc?:
Map<string, number>): BMRresult {
    const n = text.length
    const m = pattern.length

    if ( (m === 0) || m > n) {
        return { indices: [], comparisons: 0 };
    }

    let last;

    if (lastOcc === null) {
        last = buildLastOccurrence(pattern);
    } else if (lastOcc === undefined) {
        last = buildLastOccurrence(pattern);
    } else {
        last = lastOcc;
    }

    const indices: number[] = [];

    let comparisons = 0;
    let i = (m - 1);

    while (i < n) {
        let j = (m - 1);
        let k = i

        while (j >= 0) {
            comparisons++;
            if (pattern[j] !== text[k]) {
                break;
            }

            j--;
        }
    }
}

```

```

        k--;
    }

    if (j<0) {
        indices.push(i - m+1);
        i++;
    } else {
        let lastIdx: number;
        const temp = last.get(text[k]);

        if (temp === null) {
            lastIdx = -1;
        } else if (temp === undefined) {
            lastIdx = -1;
        } else {
            lastIdx = temp;
        }

        i += Math.max(1, (j - lastIdx))
    }
}

return { indices, comparisons };
}

```

Implementasi yang di atas akan menggunakan **bad character heuristic**, yang merupakan salah satu dari *heuristic* yang ada di algoritma Boyer-Moore. Tidak jauh berbeda dengan algoritma KMP, BM juga harus membentuk sebuah tabel berupa `buildLastOccurence` yang baru nanti dapat dilanjut dengan pencarian.

1. Pembuatan tabel *Last Occurence*

Tabel ini berisi setiap karakter **unik** yang ada di pola, dan *value* merepresentasikan 'posisi' di mana karakter tersebut

muncul terakhir di keseluruhan pola. Atau, dalam kata lain, huruf unik jika dilihat dari kanan pola ke arah kiri.

Untuk huruf yang tidak dalam pola, kami define dengan angka -1 saja.

2. Pencarian pola dengan Boyer-Moore

Pencarian akan dilakukan dari ujung kanan kata menuju arah kanan kata. Hal tersebut juga berlaku untuk perbandingan antara kata dengan pola. Namun untuk iterasi tetap mempertahankan mulai dari ujung kiri, hanya perbandingan yang di ujung kanan. Jika nanti terjadi ketidakcocokan, hanya ada satu perhitungan untuk menentukan banyak *shift* yaitu,

$$shift = \max(1, j - lastOcc[text[k]])$$

3. [aho-corasick.ts](#)

```
import { normalizeText } from '../shared/text-utils';

export interface AhoCorasickMatch {
  keyword: string;
  index: number;
}

export interface AhoCorasickResult {
  matches: AhoCorasickMatch[];
  comparisons: number;
}
```

```

export interface AhoCorasickNode {
    next: Map<string, number>;
    fail: number;
    output: string[];
}

function createNode(): AhoCorasickNode {
    return {
        next: new Map<string, number>(),
        fail: 0,
        output: [],
    };
}

function buildTrie(patterns: string[]): AhoCorasickNode[] {
    const trie: AhoCorasickNode[] = [createNode()];

    for (let i = 0; i < patterns.length; i++){
        const pattern = normalizeText(patterns[i]);

        if (pattern.length === 0){
            continue;
        }

        let current = 0;

```

```

    for (let j = 0; j < pattern.length; j++){
        const c = pattern[j];
        const nextNode = trie[current].next.get(c);

        if (nextNode === undefined){
            trie.push(createNode());
            trie[current].next.set(c, trie.length - 1);
            current = trie.length - 1;
        } else {
            current = nextNode;
        }
    }

    trie[current].output.push(pattern);
}

return trie;
}

function buildFailure(trie: AhoCorasickNode[]): void {
    const queue: number[] = [];

    for (const child of trie[0].next.values()){
        trie[child].fail = 0;
        queue.push(child);
    }
}

```

```

let head = 0;

while (head < queue.length){
    const current = queue[head];
    head++;

    for (const [char, child] of
trie[current].next.entries()){
        queue.push(child);
        let failure = trie[current].fail;

        while (failure !== 0 && trie[failure].next.get(char)
=== undefined){
            failure = trie[failure].fail;
        }

        const nextFailure = trie[failure].next.get(char);

        if (nextFailure !== undefined){
            trie[child].fail = nextFailure;
        } else {
            trie[child].fail = 0;
        }

        const failureOut = trie[trie[child].fail].output;

```



```

        for (let i = 0; i < failureOut.length; i++){
            trie[child].output.push(failureOut[i]);
        }
    }
}

export function precompute(patterns: string[]): AhoCorasickNode[]
{
    const trie = buildTrie(patterns);
    buildFailure(trie);
    return trie;
}

export function ahoCorasickSearch(
    text: string,
    pattern: string[],
    automation?: AhoCorasickNode[]
): AhoCorasickResult {
    const source = normalizeText(text);
    const trie = automation === undefined ? precompute(pattern) :
automation;

    const matches: AhoCorasickMatch[] = [];
    let comparisons = 0;
    let state = 0;

```

```

    for (let i = 0; i < source.length; i++){
        const c = source[i];

        while (state !== 0 && trie[state].next.get(c) ===
undefined){
            state = trie[state].fail;
            comparisons++;
        }

        comparisons++;
        const nextState = trie[state].next.get(c);

        if (nextState !== undefined){
            state = nextState;
        } else {
            state = 0;
        }

        for (let j = 0; j < trie[state].output.length; j++){
            const keyword = trie[state].output[j];

            matches.push({
                keyword,
                index: i - keyword.length + 1,
            });
        }
    }

```

```
}  
    return { matches, comparisons };  
}
```

Implementasi `aho-corasick.ts` digunakan sebagai algoritma bonus untuk mencari banyak keyword sekaligus dalam satu kali pemindaian teks. Setiap hasil pencarian direpresentasikan oleh `AhoCorasickMatch`, sedangkan hasil keseluruhan dikembalikan dalam bentuk `AhoCorasickResult` yang berisi daftar match dan jumlah comparison. Struktur utama algoritma ini adalah **AhoCorasickNode** yang memiliki tiga atribut, yaitu `next`, `fail`, dan `output`. `next` menyimpan transisi karakter ke node berikutnya, `fail` menyimpan fallback state ketika terjadi mismatch, dan `output` menyimpan keyword yang berakhir pada node tersebut.

Pada tahap awal, fungsi **buildTrie** membangun trie dari seluruh pattern. Setiap pattern dinormalisasi terlebih dahulu menggunakan `normalizeText`, lalu setiap karakternya dimasukkan ke dalam trie. Jika transisi karakter belum tersedia, node baru dibuat dan disimpan ke dalam `next`. Setelah satu pattern selesai diproses, pattern tersebut dimasukkan ke dalam `output` node terakhir. Dengan cara ini, keyword yang memiliki prefix sama dapat berbagi jalur di dalam trie.

Setelah trie terbentuk, fungsi **buildFailure** membangun failure link menggunakan queue. Setiap anak langsung dari root diberi nilai `fail = 0`. Kemudian, untuk setiap node, algoritma mengecek transisi karakter yang dimiliki node tersebut. Jika transisi tidak ditemukan pada state failure, sistem akan mengikuti failure link sampai menemukan transisi yang sesuai atau kembali ke root. Node juga mewarisi output dari failure link-nya agar pattern yang saling overlap tetap dapat terdeteksi.

Fungsi **precompute** digunakan untuk membungkus tahap pra-pemrosesan, yaitu membangun trie dan failure link sekaligus. Setelah automaton selesai dibuat, fungsi `ahoCorasickSearch` melakukan pencarian dengan membaca teks dari kiri ke kanan. Pada setiap karakter, state automaton diperbarui mengikuti transisi yang tersedia. Jika state memiliki output, maka keyword yang terdapat pada output tersebut dicatat sebagai hasil match dengan indeks awal $i - \text{keyword.length} + 1$. Hasil akhir dari fungsi ini adalah daftar keyword yang ditemukan beserta jumlah comparison selama proses pencarian.

4. rabin-karp.ts

```
import { normalizeText } from '../shared/text-utils';

export interface RKResult {
  indices: number[];
  comparisons: number;
}

export interface RKMatch {
  keyword: string;
  index: number;
}

export interface RKMultiResult {
  matches: RKMatch[];
  comparisons: number;
}

export interface RKPrecomputed {
  pattern: string;
  hash: bigint;
  power: bigint;
}

const BASE = 257n;
const MOD = 1000000007n;
```

```

function charCode(c: string): bigint {
    return BigInt(c.charCodeAt(0));
}

function normalizeHash(value: bigint): bigint {
    let result = value % MOD;

    if (result < 0n){
        result += MOD;
    }
    return result;
}

function appendHash(current: bigint, c: string): bigint {
    return normalizeHash(current * BASE + charCode(c));
}

function buildHash(text: string): bigint {
    let hash = 0n;

    for (let i = 0; i < text.length; i++){
        hash = appendHash(hash, text[i]);
    }
    return hash;
}

```

```

function buildPower(length: number): bigint {
    let power = 1n;

    for (let i = 1; i < length; i++){
        power = normalizeHash(power * BASE);
    }
    return power;
}

function rollHash(
    currentHash: bigint,
    leftChar: string,
    rightChar: string,
    power: bigint
): bigint {
    let hash = currentHash - normalizeHash(charCode(leftChar) *
power);
    hash = normalizeHash(hash);
    hash = appendHash(hash, rightChar);

    return hash;
}

function verifyAt(text: string, pattern: string, start: number):
{

```

```

    matched: boolean;
    comparisons: number;
  } {
    let comparisons = 0;

    for (let i = 0; i < pattern.length; i++){
      comparisons++;

      if (text[start + i] !== pattern[i]){
        return {
          matched: false,
          comparisons,
        };
      }
    }

    return {
      matched: true,
      comparisons,
    };
  }

export function precompute(pattern: string): RKPrecomputed {
  const normalizedPattern = normalizeText(pattern);

  return {

```



```

        pattern: normalizedPattern,
        hash: buildHash(normalizedPattern),
        power: buildPower(normalizedPattern.length),
    };
}

export function rabinKarpSearch(
    text: string,
    pattern: string,
    prepared?: RKPrecomputed
): RKResult {
    const source = normalizeText(text);
    const data = prepared === undefined ? precompute(pattern) :
prepared;
    const n = source.length;
    const m = data.pattern.length;

    if (m === 0 || m > n){
        return {
            indices: [],
            comparisons: 0,
        };
    }

    const indices: number[] = [];
    let comparisons = 0;

```

```

let textHash = buildHash(source.slice(0, m));

for (let i = 0; i <= n - m; i++){
    comparisons++;
    if (textHash === data.hash){
        const verified = verifyAt(source, data.pattern, i);
        comparisons += verified.comparisons;

        if (verified.matched){
            indices.push(i);
        }
    }

    if (i < n - m){
        textHash = rollHash(
            textHash,
            source[i],
            source[i + m],
            data.power
        );
    }
}

return {indices, comparisons};
}

export function rabinKarpMultiSearch(

```

```

    text: string,
    patterns: string[]
  ): RKMultiResult {
    const matches: RKMatch[] = [];
    let comparisons = 0;

    for (let i = 0; i < patterns.length; i++){
      const pattern = patterns[i];

      if (pattern.length === 0){
        continue;
      }

      const result = rabinKarpSearch(text, pattern);
      comparisons += result.comparisons;

      for (let j = 0; j < result.indices.length; j++){
        matches.push({
          keyword: normalizeText(pattern),
          index: result.indices[j],
        });
      }
    }

    return {matches, comparisons};
  }

```

Implementasi rabin-karp.ts digunakan sebagai algoritma bonus untuk melakukan pencocokan string berbasis hashing. Hasil pencarian

satu pattern direpresentasikan oleh `RKResult`, sedangkan pencarian banyak pattern direpresentasikan oleh `RKMultiResult`. Selain itu, terdapat struktur `RKPrecomputed` yang menyimpan pattern hasil normalisasi, nilai hash pattern, dan nilai power yang digunakan pada rolling hash.

Pada tahap awal, algoritma mendefinisikan konstanta $BASE = 257^n$ dan $MOD = 1000000007^n$ sebagai dasar perhitungan hash. Fungsi **buildHash** digunakan untuk membentuk hash dari sebuah string dengan cara membaca karakter satu per satu, lalu memperbarui hash menggunakan fungsi `appendHash`. Agar nilai hash tetap berada dalam rentang modulo dan tidak bernilai negatif, digunakan fungsi `normalizeHash`.

Konsep utama dari Rabin-Karp adalah **rolling hash**. Pada implementasi ini, fungsi `rollHash` digunakan untuk menghitung hash window berikutnya tanpa menghitung ulang dari awal. Nilai hash lama dikurangi kontribusi karakter paling kiri, kemudian karakter baru di sebelah kanan ditambahkan. Untuk mendukung proses tersebut, fungsi `buildPower` menghitung pangkat basis yang dibutuhkan untuk menghapus karakter paling kiri dari window.

Fungsi **precompute** digunakan untuk melakukan pra-pemrosesan pattern. Pattern terlebih dahulu dinormalisasi menggunakan `normalizeText`, kemudian dihitung nilai hash dan power-nya. Hasil pra-pemrosesan ini dapat digunakan kembali pada proses pencarian agar hash pattern tidak perlu dihitung berulang kali.

Pencarian utama dilakukan melalui fungsi **rabinKarpSearch**. Teks dinormalisasi terlebih dahulu, lalu algoritma menghitung hash window awal dengan panjang yang sama seperti pattern. Pada setiap posisi window, hash teks dibandingkan dengan hash pattern. Jika kedua hash sama, fungsi `verifyAt` akan melakukan pengecekan karakter satu per satu untuk memastikan bahwa kecocokan

benar-benar valid dan bukan akibat hash collision. Jika valid, indeks awal kemunculan pattern dimasukkan ke dalam indices. Setelah itu, window digeser ke kanan menggunakan rollHash hingga seluruh teks selesai diperiksa.

Selain pencarian satu pattern, file ini juga menyediakan fungsi `rabinKarpMultiSearch` untuk mencari banyak pattern. Fungsi ini menjalankan `rabinKarpSearch` untuk setiap pattern, lalu menggabungkan seluruh hasilnya ke dalam array `matches`. Dengan demikian, Rabin-Karp dapat digunakan sebagai alternatif exact matching untuk mendeteksi kata kunci judi online dengan pendekatan berbasis hash.

5. levenshtein.ts

```
import {
  extractTextCandidates,
  hasTrailingDigit,
  normalizeText,
  sameVisualGroup,
  stripTrailingDigits,
} from '../shared/text-utils';

export interface LevenshteinMatch {
  keyword: string;
  candidate: string;
  index: number;
  distance: number;
```

```

        similarity: number;
    }

    export interface LevenshteinResult {
        matches: LevenshteinMatch[];
        comparisons: number;
    }

    export interface DistanceResult {
        distance: number;
        comparisons: number;
    }

    export interface SimilarityResult {
        distance: number;
        similarity: number;
        comparisons: number;
    }

    const DEFAULT_THRESHOLD = 0.75;
    const INSERT_COST = 1;
    const DELETE_COST = 1;
    const SUBSTITUTE_COST = 1;
    const VISUAL_SUBSTITUTE_COST = 0.25;

    function substitutionCost(a: string, b: string): number {

```

```

    if (a === b){
        return 0;
    }

    if (sameVisualGroup(a, b)){
        return VISUAL_SUBSTITUTE_COST;
    }
    return SUBSTITUTE_COST;
}

function min3(a: number, b: number, c: number): number {
    return Math.min(a, Math.min(b, c));
}

function comparableCandidate(candidate: string, keyword: string):
string {
    if (!hasTrailingDigit(keyword)){
        const stripped = stripTrailingDigits(candidate);
        if (stripped.length > 0){
            return stripped;
        }
    }
    return candidate;
}

```

```

export function weightedLevenshteinDistance(a: string, b:
string): DistanceResult {
    const s1 = normalizeText(a);
    const s2 = normalizeText(b);
    const n = s1.length;
    const m = s2.length;

    if (n === 0){
        return { distance: m, comparisons: 0 };
    }

    if (m === 0){
        return { distance: n, comparisons: 0 };
    }

    let previous = new Array<number>(m + 1).fill(0);
    let current = new Array<number>(m + 1).fill(0);

    for (let j = 0; j <= m; j++){
        previous[j] = j * INSERT_COST;
    }

    let comparisons = 0;

    for (let i = 1; i <= n; i++){
        current[0] = i * DELETE_COST;

```



```

        for (let j = 1; j <= m; j++){
            comparisons++;

            const deleteCost = previous[j] + DELETE_COST;
            const insertCost = current[j - 1] + INSERT_COST;
            const replaceCost = previous[j - 1] +
substitutionCost(s1[i - 1], s2[j - 1]);

            current[j] = min3(deleteCost, insertCost,
replaceCost);
        }

        const temp = previous;
        previous = current;
        current = temp;
    }

    return {
        distance: previous[m],
        comparisons,
    };
}

export function similarityScore(a: string, b: string):
SimilarityResult {
    const result = weightedLevenshteinDistance(a, b);
    const maxLength = Math.max(a.length, b.length);

    if (maxLength === 0){

```

```

        return {
            distance: 0,
            similarity: 1,
            comparisons: result.comparisons,
        };
    }

    const similarity = Math.max(0, 1 - result.distance /
maxLength);
    return {
        distance: result.distance,
        similarity,
        comparisons: result.comparisons,
    };
}

export function levenshteinSearch(
    text: string,
    keywords: string[],
    threshold = DEFAULT_THRESHOLD
): LevenshteinResult {
    const candidates = extractTextCandidates(text);
    const matches: LevenshteinMatch[] = [];
    let comparisons = 0;

    for (let i = 0; i < candidates.length; i++){

```

```

    for (let j = 0; j < keywords.length; j++){
        const keyword = keywords[j];

        if (keyword.length === 0){
            continue;
        }

        const compared =
comparableCandidate(candidates[i].word, keyword);

        if (compared.length === 0){
            continue;
        }

        const result = similarityScore(compared, keyword);
        comparisons += result.comparisons;

        if (result.similarity >= threshold){
            matches.push({
                keyword,
                candidate: candidates[i].word,
                index: candidates[i].index,
                distance: result.distance,
                similarity: result.similarity,
            });
        }
    }

```

```

    }
}
return {matches, comparisons};
}

```

Implementasi levenshtein.ts digunakan untuk mendukung fitur fuzzy matching, yaitu pencocokan kata yang tidak sama persis dengan keyword, tetapi masih memiliki tingkat kemiripan yang cukup tinggi. Fitur ini digunakan untuk menangani variasi penulisan atau obfuscation yang sering ditemukan pada konten judi online, misalnya penggantian huruf dengan angka yang terlihat mirip seperti O menjadi 0, A menjadi 4, atau I menjadi 1. Hasil pencarian direpresentasikan oleh **LevenshteinMatch**, yang menyimpan keyword, kandidat kata dari teks, indeks kemunculan, distance, dan similarity.

Pada implementasi ini, digunakan beberapa konstanta bobot, yaitu INSERT_COST, DELETE_COST, dan SUBSTITUTE_COST bernilai 1, serta VISUAL_SUBSTITUTE_COST bernilai 0.25. Nilai substitusi visual dibuat lebih kecil karena perubahan karakter yang mirip secara visual dianggap sebagai manipulasi ringan, bukan perubahan yang sepenuhnya berbeda. Fungsi **substitutionCost** akan mengembalikan biaya 0 jika karakter sama, 0.25 jika kedua karakter berada pada kelompok visual yang sama, dan 1 jika karakter benar-benar berbeda.

Fungsi **weightedLevenshteinDistance** menghitung jarak antara dua string menggunakan dynamic programming. Sebelum dihitung, kedua string dinormalisasi terlebih dahulu menggunakan normalizeText agar pencocokan lebih konsisten. Untuk menghemat memori, implementasi hanya menggunakan dua array, yaitu previous dan current, bukan menyimpan seluruh matriks. Pada setiap pasangan karakter, algoritma menghitung tiga kemungkinan operasi,

yaitu delete, insert, dan replace. Nilai minimum dari ketiga operasi tersebut digunakan sebagai nilai distance pada posisi terkait.

Setelah distance diperoleh, fungsi **similarityScore** mengubah jarak tersebut menjadi skor kemiripan menggunakan rumus $1 - \text{distance} / \text{maxLength}$. Nilai similarity berada pada rentang 0 sampai 1, dengan nilai yang semakin mendekati 1 berarti kedua string semakin mirip. Nilai ini kemudian dibandingkan dengan threshold default 0.75 untuk menentukan apakah kandidat dianggap sebagai hasil fuzzy match atau tidak.

Fungsi **levenshteinSearch** menjadi fungsi utama untuk melakukan pencarian fuzzy. Teks terlebih dahulu dipecah menjadi kandidat kata menggunakan `extractTextCandidates`, kemudian setiap kandidat dibandingkan dengan keyword. Sebelum dibandingkan, fungsi `comparableCandidate` dapat menghapus angka di akhir kandidat jika keyword tidak memiliki angka di akhir. Hal ini membuat variasi seperti GACOR99 tetap dapat dibandingkan terhadap keyword GACOR, sementara pola lengkap <kata><angka> tetap dapat ditangani oleh RegEx. Jika similarity kandidat terhadap keyword lebih besar atau sama dengan threshold, kandidat tersebut dicatat sebagai hasil match.

4.1.2 Algoritma *Regular Expression* (RegEx)

```
export interface RegexMatch {  
    word: string;  
    digits: string;  
    fullMatch: string;
```

```

        index: number;

    }

    export interface RegexResult {

        matches: RegexMatch[];

        comparisons: number;

    }

    // Map number to alphabet that looks similar

    const LOOKALIKE: Record<string, string> = {

        '0': 'O', '1': 'I', '3': 'E', '4': 'A', '5': 'S', '6': 'G',
        '7': 'T',

    };

    function normalizeLookalikes(text: string): string {

        return text

            .toUpperCase()

            .split('')

            .map((c) => LOOKALIKE[c] ?? c)

            .join('');
    }

```

```

    }

    const RAW_PATTERN = /([A-Za-z][A-Za-z]*)(\d{2,})/g;

    function extractMatches(source: string, originalText: string,
offset = 0): RegExpMatch[] {

        const pattern = new RegExp(RAW_PATTERN.source, 'g');

        const results: RegExpMatch[] = [];

        let m: RegExpExecArray | null;

        while ((m = pattern.exec(source)) !== null) {

            results.push({

                word: m[1],

                digits: m[2],

                fullMatch: originalText.slice(m.index + offset, m.index +
offset + m[0].length),

                index: m.index + offset,

            });

        }

        return results;

    }

```

```

export function regexSearch(text: string): RegexResult {

    let comparisons = text.length;

    const seen = new Set<number>();

    const matches: RegexMatch[] = [];

    // Matched the original text

    for (const m of extractMatches(text, text)) {

        seen.add(m.index);

        matches.push(m);

    }

    // Matched the normalized text (G4C0R99 = GACOR99)

    const normalized = normalizeLookalikes(text);

    comparisons += normalized.length;

    for (const m of extractMatches(normalized, text)) {

        if (!seen.has(m.index)) {

            seen.add(m.index);

            matches.push(m);

        }

    }

}

```



```
return { matches, comparisons };  
  
}
```

Modul `regex-match.ts` mendefinisikan struktur hasil melalui *interface* `RegexMatch` dan `RegexResult`, lalu menjalankan pencocokan berbasis regex dengan dua tahap, yaitu tahap pengecekan untuk teks asli dan teks yang telah dinormalisasi. *Map* LOOKALIKE digunakan untuk menyetarakan karakter yang mirip secara visual (misalnya 0→O, 1→l) melalui fungsi `normalizeLookalikes`, sehingga variasi penulisan seperti “G4C0R99” dapat terdeteksi sebagai “GACOR99”. Pola inti disimpan pada `RAW_PATTERN` yang mengekstrak rangkaian huruf diikuti minimal dua digit, sehingga hasil cocok dipetakan ke `word`, `digits`, `fullMatch`, dan `index`.

Fungsi `extractMatches` menjalankan regex terhadap *source* dan membangun daftar hasil dengan posisi absolut terhadap teks asli. Di dalam `regexSearch`, pencocokan pertama dilakukan langsung pada teks asli untuk menangkap kemunculan *literal*, lalu pencocokan kedua dilakukan pada teks yang telah dinormalisasi. *Set* seen dipakai untuk menghindari duplikasi ketika pola yang sama terdeteksi di posisi indeks yang sama. Nilai *comparisons* diakumulasikan dari panjang teks asli dan teks hasil normalisasi sebagai indikator jumlah pemeriksaan. Hasil akhirnya berupa objek `{matches, comparisons}`, berisi hasil temuan lengkap beserta jumlah pengecekan yang dilakukan.

4.2 Penjelasan Tata Cara Penggunaan Extension

Extension dijalankan melalui beberapa tahapan utama yang dijabarkan lebih lanjut di bawah.

1. Instalasi *Extension*

- Buka halaman `chrome://extensions/` atau halaman extension pada *browser* chromium lainnya.
- Aktifkan Developer mode.
- Klik Load unpacked lalu pilih folder hasil build *extension*.
- Pastikan ikon extension muncul di *toolbar browser*.

2. Penggunaan di Halaman Web

- Buka halaman web yang ingin dianalisis.
- Tekan tombol "*Scan Page*" dan sistem memindai konten halaman.
- Atur fitur tambahan seperti blur jika diperlukan.
- *Extension* akan menampilkan jumlah kata yang terindikasi judul untuk setiap algoritma, juga waktu pemindaian. *Extension* juga akan melakukan blur pada kata-kata terkait apabila konfigurasi blur diatur.

4.3 Hasil Pengujian

Seluruh pengujian menggunakan *file* HTML custom dengan pengecualian *test case* yang ke-5 dengan menguji langsung dari *google search* dari kata 'GACOR99'.

A. Test1.html

gacor999

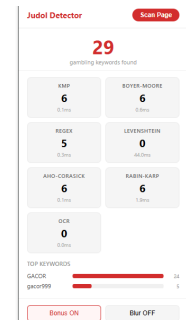
gacor999 muncul sekali di satu text node.

gacor999 dipisah elemen, jadi dua text node.

Variasi lain: gacor999 dipisah jadi tiga text node.

3 kemunculan dalam satu text node: gacor999 dan gacor999 lagi dan gacor999 lagi.

gacor999		
Occurrences: 3		
KMP	0	0.10ms
Boyer-Moore	0	0.60ms
Regex	3	0.30ms
Levenshtein	0	44.00ms
Aho-Corasick	0	0.10ms
Rabin-Karp	0	1.90ms
OCR	0	0.00ms



B. Test2.html

game	provider	fitur
Gates of Olympus	Pragmatic	scatter maxwin
Sweet Bonanza	Pragmatic	freespins wild
Mahjong Ways	PG Soft	bonus jackpot

di list

- togel online terpercaya
- slot gacor hari ini
- mahjong ways maxwin
- pragmatic play scatter

1. daftar dulu
2. deposit minimal
3. spin slot dan raih jackpot

nested dalam banget

fitur wild dan scatter bisa bikin maxwin

banyak keyword dalam satu paragraf

slot togel mahjong pragmatic bonus freespins jackpot scatter wild gacor semua ada di sini hoki menanti zeus olympus gates bonanza maxwin

di blockquote

"raih gates of olympus scatter bonanza maxwin bers
— admin gacor

keyword muncul 3x dalam 1 node

slot malam ini slot siang ini slot pagi ini semuanya gacor

SCATTER		
Occurrences: 4		
KMP	1	1.00ms
Boyer-Moore	1	0.40ms
Regex	0	1.10ms
Levenshtein	0	175.70ms
Aho-Corasick	1	0.60ms
Rabin-Karp	1	5.90ms
OCR	0	0.00ms

C. Test3.html

coba **slot** online sekarang

raih **jackpot** di setiap putaran

aktifkan **bonus** eksklusif member baru

substitusi 1 huruf (harusnya kedeteksi)

situs **gacor** pilihan pemenang

fitur **scatter** membayar berlipat

spin **fr3espin** tanpa henti

daftar di **10gel**

huruf besar FREESPIN = fr3espin

menang di **Ga**

fitur **MaxWin**

daftar **TOGEL**

main di **sLoT**

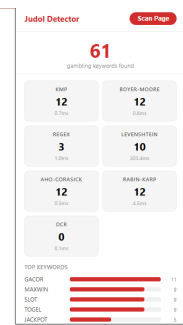
raih **JACKPOT** sekarang

keyword ada di tengah kata panjang (harusnya kedeteksi)

daftar di **slot gacor 88** sekarang

main **pragmaticplay** dan menang

promo **fre spins 100x** member baru



D. Test4.html

situs **gacor maxwin slot jackpot togel scatter**
wild bonus **fre spins**

di atribut HTML (harusnya TIDAK kedeteksi)

 **jackpot togel**

teks biasa terlihat user (HARUS kedeteksi)

mainkan **slot** dan menangkan **jackpot** besar

fitur **scatter** dan **wild** ada di semua game **pragmatic**

dapatkan **bonus** dan **fre spins** dari situs **gacor**

tiap huruf dipisah span (harusnya TIDAK kedeteksi)

SLOT

kata di atas dipecah per huruf jadi 1 karakter

spasi ekstra di sekitar keyword (HARUS kedeteksi)

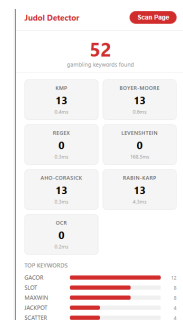
selamat datang di situs **gacor** terpercaya

menangkan **maxwin** setiap malam

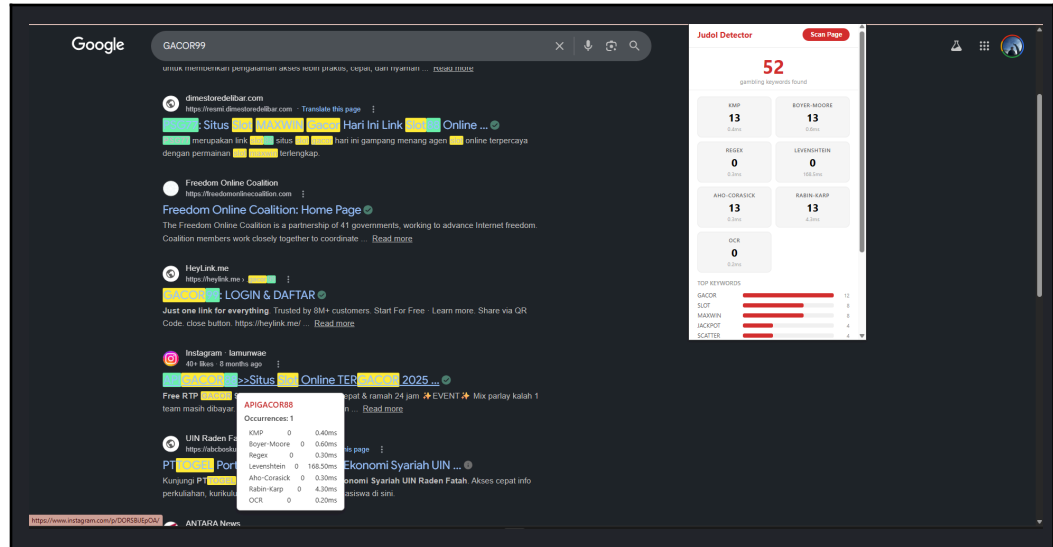
keyword di awal dan akhir paragraf

gacor adalah situs terbaik untuk semua permainan **slot**

daftar sekarang dan raih **maxwin**



E. GACOR99 on Google Search



4.4 Analisis Hasil Pengujian

Setelah melakukan pengujian dari 5 (lima) *testcase* yang berbeda, dapat disimpulkan hasil dari masing-masing algoritma termasuk pemilihan kata untuk di-*highlight* oleh *extension* sudah sangat baik yang dimana dapat memberikan gambaran yang jelas perbedaan dari hasil algoritma yang berdasarkan *exact match* (e.g. KMP, BM, etc.), *fuzzy* (e.g. Levenshtein), dan RegEx dengan warna yang berbeda-beda.

Performa dari masing-masing algoritma cukup efisien, menimbang memang komputasi dan *logic* yang digunakan dari algoritma tidak seberat itu, termasuk *data* yang diproses bersifat 'tabular'. Namun, Levenshtein memiliki sedikit ketidakefisienan dalam proses algoritmanya, mengingat jenis pencarian tidak bersifat eksak.

V. Kesimpulan dan Saran

5.1 Kesimpulan

Berdasarkan proses implementasi dan pengujian yang telah dilakukan pada Chromium browser extension Judol Detector, dapat ditarik beberapa kesimpulan sebagai berikut.

- Algoritma pattern matching telah berhasil diterapkan untuk mendeteksi konten yang mengandung unsur judi online pada halaman web. Algoritma Knuth-Morris-Pratt (KMP) dan Boyer-Moore (BM) digunakan untuk melakukan exact matching terhadap daftar kata kunci dari keywords.txt, sedangkan Regular Expression (RegEx) digunakan untuk mendeteksi pola <kata><angka> seperti GACOR99, SLOT777, atau MAXWIN88.
- Sistem mampu menangani variasi penulisan kata melalui pendekatan fuzzy matching menggunakan Weighted Levenshtein Distance. Dengan pemberian bobot yang lebih kecil pada substitusi karakter yang mirip secara visual, sistem dapat mendeteksi bentuk manipulasi seperti penggunaan angka sebagai pengganti huruf, misalnya H0KI, G4COR, atau variasi lain yang masih memiliki kemiripan tinggi terhadap kata kunci.
- Implementasi algoritma bonus Aho-Corasick dan Rabin-Karp berhasil digunakan sebagai alternatif pencocokan string. Aho-Corasick cocok untuk pencarian banyak keyword sekaligus karena membangun automaton dari seluruh pattern, sedangkan Rabin-Karp menggunakan rolling hash untuk membandingkan pattern dengan potongan teks secara efisien.

5.2 Saran

Berdasarkan batasan sistem yang ada saat ini, terdapat beberapa saran pengembangan yang dapat dilakukan untuk menyempurnakan aplikasi ke depannya.

- Peningkatan performa OCR dapat dilakukan karena proses OCR membutuhkan waktu lebih lama dibandingkan algoritma string matching biasa. Hal ini terjadi karena OCR harus membaca dan memproses gambar terlebih dahulu sebelum teks hasil ekstraksi dapat dicocokkan dengan keyword. Optimasi dapat dilakukan dengan membatasi jumlah gambar yang dipindai, memprioritaskan gambar berukuran besar, atau menyediakan toggle OCR agar pengguna dapat mengaktifkannya hanya saat diperlukan.
- Sistem deteksi dapat dikembangkan agar mampu membaca lebih banyak jenis konten web modern. Saat ini, pemindaian utama dilakukan terhadap text node pada DOM halaman. Ke depannya, dukungan terhadap iframe, shadow DOM, teks pada atribut tertentu, atau konten yang dirender secara dinamis dapat ditingkatkan agar hasil deteksi menjadi lebih menyeluruh.
- Daftar keyword dan aturan normalisasi dapat diperluas agar sistem lebih adaptif terhadap variasi konten judi online. Penambahan keyword baru, perluasan kelompok karakter visual serupa, serta penyesuaian threshold pada Weighted Levenshtein Distance dapat membantu meningkatkan akurasi deteksi dan mengurangi kemungkinan false positive maupun false negative.

VI. Lampiran

Repository dan Video program pada Tugas Besar 3 ini dapat diakses melalui tautan berikut:

Link Repository Github : [hsbu/Tubes3_GDP-UOB-OCBC](https://github.com/hsbu/Tubes3_GDP-UOB-OCBC)

Link Video Youtube : <https://youtu.be/XtOEyK4VE8o>

Adapun berikut tabel pembagian tugas:

No	Mahasiswa	NIM	Tugas
1	Muhamad Hasbullah Faris	18223014	1. Mengerjakan laporan 2. Mengimplementasikan alur kerja sistem 3. Mengimplementasikan popup 4. Mengimplementasikan tooltip 5. Mengimplementasikan algoritma RegEx
2	Stanislaus Ardy Bramantyo	18223057	1. Mengerjakan laporan 2. Main testing di situs https://klikgacor99.com/. 😊 3. Testing program dengan <i>test case</i> yang ada. 4. Mengerjakan Algoritma KMP. 5. Mengerjakan Algoritma BM. 6. Mengerjakan UI dan implementasinya sebagian

3	Nazwan Muttaqin	Siddqi	18223066	1. Mengerjakan laporan 2. Mengimplementasi Weighted Levenshtein 3. Mengimplementasi Algoritma Aho-Corasick 4. Mengimplementasi Algoritma Rabin-Karp 5. Mengimplementasi bonus Blur 6. Mengimplementasi OCR
---	--------------------	--------	----------	---

Terakhir, berikut tabel spesifikasi dari Tugas Besar 3 ini:

No	Poin	Ya	Tidak
1	Extension berhasil di-build dan di-load tanpa kesalahan pada chromium browser dan dikembangkan dengan TypeScript	✓	
2	KMP dan Boyer-Moore diimplementasikan from scratch	✓	
3	Regex handle format <kata><angka> dan berbagai edge case	✓	
4	Pencarian KMP & BM membaca keyword.txt secara iteratif dan tidak menggunakan built-in search function atau library eksternal	✓	

5	Exact matching dan fuzzy matching berjalan benar	✓	
6	Elemen DOM terdeteksi diberi highlight dan terhapus saat rescanning	✓	
7	Tooltip muncul saat hover dengan informasi keyword, algoritma, kemunculan, dan waktu eksekusi	✓	
8	Popup menampilkan statistik realtime (total keyword, perbandingan, waktu eksekusi, jumlah match)	✓	
9	[Bonus] Membuat Video	✓	
10	[Bonus] Implementasi Algoritma Aho-Corasick dan Rabin Karp	✓	
11	[Bonus] Implementasi Censorship / Blur Teks	✓	
12	[Bonus] Implementasi Optical Character Recognition pada Gambar	✓	

VII. Pernyataan Kejujuran

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.



Muhamad Hasbullah Faris



Stanislaus Ardy Bramantyo



Nazwan Siddqi Muttaqin

VIII. Daftar Pustaka

- **Chrome Extension Documentation**
<https://developer.chrome.com/docs/extensions/>
- **Tesseract.js**
<https://tesseract.projectnaptha.com/>
- **Aho-Corasick**
<https://www.geeksforgeeks.org/dsa/aho-corasick-algorithm-pattern-searching/>
- **Rabin Karp**
<https://www.geeksforgeeks.org/dsa/rabin-karp-algorithm-for-pattern-searching/>